

Intelligent Wellbore Fracture & Facies Analysis

Delivery plan · app 27 September – 1 November 2026 · Ahmed’s documentation and Mariam’s slides before 31 December 2026

The project

The project is an AI-based software system for the automated interpretation of wellbore image logs and conventional well logs.

The main goal is to analyze FMI (Formation MicroImager) wellbore images together with well-log data to automatically detect geological fractures, determine their depth and orientation, visualize them using tadpole plots, and classify subsurface intervals into different facies/zones.

A user creates an account, uploads data for a well, and clicks **Analyze Well**. The system then automatically processes the well through the complete analysis pipeline: fracture detection, fracture geometry, tadpole visualization, well-log integration, and facies/zone classification. The results are stored under the user's account and can be viewed later through the dashboard, along with a generated well analysis report.

Project pipeline

Each step feeds the next · 27 Sep – 1 Nov 2026

01 Sign up / Log in / Reset password	07 Facies / Zone Classification
02 FMI Image	08 Database
03 Fracture Detection	09 AI Agent
04 Fracture Geometry	10 Interactive Dashboard
05 Tadpole Visualization	11 My Reports — this user only
06 Well-Log Integration	12 Download report

Week 1 — Data contract and empty app

27 Sep – 3 Oct · Both people know the real files, and an empty app runs locally.

Mariam — Dataset inspection

27 Sep – 30 Sep

- List every file for 16A, 16B, and 56-32: FMI format, size, depth range, resolution.
- Decide whether fracture labels are pixel masks, polylines, or a depth / dip / azimuth table.
- Decide whether facies labels exist. Write yes or no and the real class names. Do not invent classes.
- Write the data contract: how depth is stored, how images align to logs, and the train / test well split.
- Place raw files under data/raw/.

Mariam — Login, signup, and dashboard shell

1 Oct – 3 Oct

- Agree JSON with Ahmed on **1 Oct**.
- Sign up (name, email, password), log in, and log out.
- **Forgot password:** enter the account email, set a new password, then log in with it.
- After login, show the dashboard: well selector, status, FMI panel, stats, facies, tadpole, logs, chat.
- Use placeholder data that matches the JSON.
- Show **Run Well Analysis** even if the job is not wired yet.
- Send a logged-out visitor to the log in page.

Ahmed — Backend skeleton and accounts

27 Sep – 3 Oct

- Create the FastAPI app, SQLite, and the module folders.
- User table and endpoints: sign up, log in, log out, current user, reset password. Store a password hash. Reset replaces the hash for that email.
- Stub the later endpoints: upload, analysis, fractures, facies, logs, visualizations, chat, my reports, download.
- Agree JSON with Mariam by **1 Oct**.
- Add a health check and a local run command.

Checkpoint · 3 Oct — Data contract is written, the API runs, a new account can sign up, that account can reset its password, and the dashboard appears only after login.

Week 2 — See the well

4 Oct – 10 Oct · Upload one well and see the FMI image and logs against depth.

Mariam — Preprocessing

4 Oct – 7 Oct

- Read FMI as the contract describes. Normalize, handle gaps, and tile large images.
- Save training samples. Keep this code separate from the model.
- By **7 Oct**, give Ahmed the inference signature: image tile in, fracture trace or mask out. The body may still be a stub.

Mariam — FMI and upload screen

8 Oct – 10 Oct

- Upload FMI and log files, then select the well.
- FMI viewer with a depth scale.
- Analysis status bound to the status API.

Ahmed — Loaders and depth

4 Oct – 10 Oct

- FMI reader and well-log reader for the columns that actually exist.
- Map an image row to measured depth from the file metadata.

- Store the upload on the logged-in user. Status is idle, running, done, or failed. Another user's well id is not returned.
- Align logs to the FMI depth range. Missing logs stay missing.
- Image and log endpoints ready by **8 Oct**.

Checkpoint · 10 Oct — One real well opens with the image and logs on the same depth axis.

Week 3 — Fractures

11 Oct – 17 Oct · Detect fractures, compute geometry, show them on the image and in a table.

Mariam — Fracture model

11 Oct – 15 Oct

- Train the model chosen in week 1. Prefer U-Net if pixel or trace labels exist. If labels are only a fracture table, train a depth-window detector and say so in the model note.
- Metrics: precision, recall, F1, and IoU where masks exist. The test well is held out.
- Export weights to `models/fracture_model/` with a short model card: data, split, metrics, known failures.
- Do not add a fracture-type head unless type labels are real.

Mariam — Fracture screen

16 Oct – 17 Oct

- Draw detected traces on the FMI image.
- Table: count, depth, dip, azimuth, type, confidence.
- **Run Well Analysis** starts the job and refreshes when status is done.

Ahmed — Geometry and storage

11 Oct – 17 Oct

- Call Mariam's model from the analysis job. Keep inference separate from geometry.
- For each trace store measured depth, dip, azimuth, and confidence. Store TVD only if the file has it. Store type only if it is labeled.
- Save wells, fractures, and sessions in SQLite, each row tied to the user who ran the analysis.
- Fractures API filters by depth and is stable by **15 Oct**.

Checkpoint · 17 Oct — One analyzed well shows overlays and a table that matches the database. If training slips, the overlay moves to 18–19 Oct.

Week 4 — Tadpole, logs, and facies

18 Oct – 24 Oct · Orientation plot, logs, and zones.

Mariam — Tadpole, logs, and zones

18 Oct – 21 Oct

- Tadpole: zoom, filter by depth, filter by type, click for fracture details.
- Well-log tracks versus depth.

- Facies intervals beside the logs, including an empty state for “labels unavailable.”

Mariam — Facies model

22 Oct – 24 Oct

- Depth-aligned features: fracture density, orientation stats, simple image texture, plus the available logs.
- If zone labels exist, train one classifier (Random Forest or XGBoost).
- Metrics: accuracy, precision, recall, F1, confusion matrix, on the held-out well when labels cover it.
- If labels do not exist, deliver the feature table and “facies unavailable.” No invented class names.
- Connect the zones panel to the real API on **24 Oct**.

Ahmed — APIs and the analysis job

18 Oct – 24 Oct

- Tadpole payload ready **19 Oct**: depth, dip, azimuth, id, type.
- Log series API ready **19 Oct**.
- Facies API: start depth, end depth, class, confidence, or an explicit unavailable flag.
- One job runs preprocess, detect, geometry, features, facies, and save.
- Report numbers come from the database: well info, fracture count, orientation stats, facies summary, confidences.
- Save the report on the analysis owner. The list returns only the caller’s rows.

Checkpoint · 24 Oct — The full picture is on screen: image, table, tadpole, logs, and zones.

Week 5 — Assistant, report, and demo

25 Oct – 1 Nov · Grounded chat, a private PDF, and a rehearsed demo. No new science after 28 Oct.

Ahmed — Agent and PDF

25 Oct – 28 Oct

- Tools only: analyze well, detect fractures, geometry, depth, tadpole data, classify facies, query wells, query results, generate report.
- Tools read the database and only the logged-in user’s rows. Counts and “highest fracture density” are computed.
- A missing field returns unavailable.
- Download returns a PDF the caller owns. Another user’s report id is not found.
- Chat and report payloads ready **26 Oct**.

Mariam — Chat, My Reports, and evaluation

25 Oct – 28 Oct

- **25–26 Oct**: chat scoped to the selected well. **My Reports** lists only this account. Each report opens in the app and has **Download**.
- **27–28 Oct**: one-page evaluation — fracture metrics on the test well, facies metrics or “no labels,” and failure examples.
- Check five agent answers against the database: count in a depth window, dip filter, highest density, explain one fracture, generate a report.

- Fix empty states and errors on upload and failed analysis.

Both — Freeze and demo

29 Oct – 1 Nov

- **28 Oct** — feature freeze.
- **29 Oct** — demo on a clean database with two accounts. Account B cannot see or download account A's report.
- **30 Oct** — fix only demo-blocking bugs.
- **31 Oct** — second rehearsal and defense screenshots.
- **1 Nov** — local demo. README explains install, where to put data, and how to run the backend and frontend.

Demo script

Rehearse this on the held-out well the night before. Use one of 16A(78)-32, 16B(78)-32, or 56-32 — the well that was not used to train the fracture model. Write the real depth window, fracture count, and one fracture id on a card. Every number said out loud must match the screen. If a value was not computed, the assistant says it is unavailable.

1. Open the problem

Say what the system is: an interpretation workflow for FMI images and conventional logs. It detects fractures, finds depth, dip, and azimuth, draws a tadpole, and classifies facies where labels exist. Then say the demo is one unseen well, not a slide of training accuracy.

2. Account

Create an account on a clean database. Log out. Use **Forgot password**, set a new password, and log in with that password. The dashboard is closed until login succeeds.

3. Load the well

Select or upload that well's FMI image and the log file. Point to the depth range on the FMI viewer before analysis, so the committee sees the raw image first. Name the log curves that actually loaded. Do not claim a curve that is missing.

4. Analyze Well

Click **Analyze Well**. While the status runs, walk the pipeline in order: fracture detection, geometry, tadpole, well-log alignment, facies, then save. When status is done, do not describe results that are not on the screen yet.

5. Fractures on the image

Show the FMI image with traces drawn on it. Scroll to one clear fracture. Read its measured depth from the table, then show that the same trace sits at that depth on the image. Read dip, azimuth, and confidence from the row. If type or TVD is empty, say the file did not provide it.

6. Tadpole

Open the tadpole plot on the same depth range. Zoom in. Filter to dip greater than 60 degrees and show that the table and the plot change together. Click one tadpole and show that its depth, dip, and azimuth are the same fracture as in the table.

7. Logs and zones

Show the well logs against the same depth axis as the fractures. If facies intervals exist, point to one interval and its confidence. If they do not, leave the empty state on screen and say zone labels were not in the dataset.

8. Assistant, from the stored well

Ask these in the chat, one at a time, and check each answer against the table before moving on:

- How many fractures were detected on this well?
- How many fractures are between the two depths written on the card?
- Which fractures have dip greater than 60 degrees?
- Which depth interval has the highest fracture density?
- Explain fracture number N: depth, dip, azimuth, type if stored, and confidence.
- What is the true vertical depth of that fracture?

The last answer is “unavailable” unless TVD was stored. The count and the dip list must equal the filtered table, not a rounded guess.

9. Report

Ask the assistant to generate the report. Open **My Reports**. Open that report in the app and show the same fracture count, orientation summary, and facies result as the dashboard. Download the PDF and show those same figures on the first pages.

10. The report belongs to this account

Log out. Log in as a second account. **My Reports** does not list the first report, and that report cannot be opened or downloaded. Log back into the first account and show the report is still there.

Ahmed — Documentation

2 Nov – 31 Dec 2026

Ahmed writes this after the 1 November demo. He must finish it by **31 December 2026**.

Mariam — Presentation slides

2 Nov – 30 Dec 2026

Mariam prepares the presentation slides after the 1 November demo. She must finish them before **31 December 2026**.